SmartWallet

Personal Finance Manager

MERN Stack Edition (MongoDB, Express.js, React, Node.js)

1. Project Overview

SmartWallet is a personal finance management platform designed to help individuals organize their finances, monitor spending habits, and manage their monthly budgets.

Many people struggle to maintain a clear understanding of their income, expenses, and savings. Daily purchases, recurring bills, and unexpected costs can make it difficult to track where money is going and determine whether spending remains within a planned budget.

SmartWallet addresses these challenges by providing a centralized platform where users can record financial transactions, categorize expenses, define monthly budgets, and monitor savings goals.

The application also integrates artificial intelligence and Optical Character Recognition (OCR) to simplify expense recording. Instead of manually entering every purchase, users can upload photos of shopping receipts. The system extracts relevant transaction information and uses AI to suggest an appropriate expense category. Users can review and correct the extracted information before saving the transaction.

The project aims to achieve three main objectives:

- **Simplify personal finance tracking:** Allow users to record and organize income and expenses through a clear and accessible interface.
- **Improve financial awareness:** Combine dashboards and budget monitoring with AI-generated insights that explain historical spending patterns and suggest practical improvements.
- **Reduce manual data entry with AI:** Use receipt recognition and intelligent categorization to make expense recording faster and more convenient.

The final application should support financial tracking, monthly budgets, savings goals, receipt OCR, and AI-generated monthly reports that help users understand recorded spending patterns.

SmartWallet is a personal finance tracking platform, not a banking application. It does not require bank account connections, money transfers, or payment processing.

2. Users and Roles

SmartWallet supports one primary user role.

User

Users are individuals who want to organize and monitor their personal finances. They should be able to:

- Create an account and manage their personal profile.
- Record income and expense transactions.
- Organize transactions into financial categories.
- Define and monitor monthly budgets.
- Create personal savings goals and track their progress.
- View financial summaries and spending patterns.
- Upload receipts to automatically extract transaction information.

- Review and correct AI-extracted information before saving transactions.
- Access their financial history and monitor changes over time.

Each user should have a private financial environment containing their transactions, budgets, categories, and savings goals.

The platform does not require banking institutions, financial advisors, merchants, or administrative roles for its core functionality.

3. Functional Requirements

3.1 Must Have: Required Features

The following features are essential for SmartWallet to function as a complete personal finance management platform.

A. Authentication and User Management

The platform must provide secure authentication and personal account management.

- Users must be able to register and log in using email and password.
- Users should be able to view and update their basic profile information.
- Users should be able to select their preferred currency for displaying financial amounts.
- Each user must have access only to their own transactions, budgets, and savings goals.
- The application must protect personal financial information and prevent unauthorized access.

The core application may operate using a single currency per user without requiring currency conversion or exchange-rate integration.

B. Income and Expense Management

SmartWallet must allow users to record and manage their financial transactions. The application should support two primary transaction types:

- **Income:** Money received, such as salaries, freelance earnings, allowances, or other sources.
- **Expense:** Money spent on purchases, services, bills, or other financial activities.

Users should be able to:

- Create income and expense transactions manually.
- Record relevant transaction information, including amount, date, category, and description.
- Edit or delete existing transactions.
- View their transaction history.
- Filter transactions by type, category, or date range.
- Search for transactions using relevant information.
- View total income, total expenses, and their financial balance for a selected period.

All recorded transactions should contribute to the application's financial summaries and budget calculations.

C. Transaction Categories

The platform must support categories to help users organize their income and expenses. Examples include:

Category Type	Example Categories
Income	Salary, Freelance, Gifts, Other Income
Expense	Groceries, Transportation, Housing, Entertainment, Shopping, Healthcare

The application should:

- Provide a set of default income and expense categories.
- Allow users to create custom categories.
- Allow users to edit or remove their custom categories when appropriate.
- Associate each transaction with a category.
- Use categories to organize financial summaries and spending analysis.

Categories should remain consistent across transaction tracking, budgeting, and dashboard reporting.

D. Monthly Budget Management

Users must be able to create and monitor monthly spending budgets. The platform should allow users to:

- Define a total monthly spending budget.
- Set optional spending limits for individual expense categories.
- View how much of each budget has been used.
- See the remaining available budget.
- Identify when spending approaches or exceeds a defined limit.
- Review budgets for previous months.
- Update budget limits when necessary.

Budget calculations must use recorded expense transactions for the relevant month and categories.

For example, a user who sets a monthly groceries budget of 2,000 MAD and records 1,400 MAD in grocery expenses should see that 70% of the budget has been used, with 600 MAD remaining.

The application should distinguish between the user's overall monthly budget and any category-specific budgets to avoid misleading calculations.

E. Personal Savings Goals

SmartWallet must allow users to define and track personal savings goals. Examples include saving for a laptop, vacation, emergency fund, or other personal objective. Users should be able to:

- Create savings goals with a name and target amount.
- Define an optional target date.
- Record contributions toward each goal.
- View the amount saved and the remaining target.

- Update or remove existing goals.
- Mark goals as completed when their targets are reached.

Savings progress should reflect recorded contributions rather than assuming all unspent income is saved. Savings contributions must be recorded separately from ordinary expenses: allocating money to a goal does not itself count as spending. Financial summaries, balances, and savings progress must use consistent calculations without double-counting contributions.

Savings goals are planning and tracking tools. The application does not need to transfer or hold money.

F. Financial Dashboard

SmartWallet must provide a dashboard presenting a clear overview of the user's financial situation. The dashboard should include:

- Total income for the selected period.
- Total expenses for the selected period.
- Remaining balance based on recorded transactions.
- Monthly budget usage.
- Spending distribution by category.
- Recent transactions.
- Savings goal progress.

The dashboard should help users quickly understand their financial activity without manually reviewing individual transactions. Financial summaries and visualizations should update when users add, edit, or delete relevant records.

G. Receipt Upload and OCR Processing

SmartWallet must allow users to upload photos of shopping receipts to simplify expense recording. The application should support the following workflow:

- 1. The user uploads an image of a shopping receipt.
- 2. The application processes the image using OCR to recognize printed text.
- 3. The system identifies relevant transaction information from the extracted text.
- 4. AI suggests an appropriate expense category.
- 5. The application presents the extracted information for user review.
- 6. The user corrects or completes any missing information.
- 7. The confirmed information is saved as an expense transaction.

The receipt processing functionality should attempt to extract:

Information	Expected Result
Merchant	Name of the store or business, when available
Date	Purchase date
Total Amount	Final amount paid

Information	Expected Result
Currency	Currency when identifiable
Suggested Category	Appropriate expense category based on receipt information

The application must allow users to correct incorrectly recognized information before saving it. Users should also be able to enter transaction information manually when OCR processing fails or produces incomplete results.

Receipt recognition must support transaction creation, not automatically save expenses without user confirmation.

H. AI Monthly Financial Analysis

SmartWallet must allow users to select a month and generate an AI-powered financial report from recorded transactions, budgets, and savings information. Users should be able to:

- Select a month for analysis.
- Review monthly income, expenses, budget performance, and highest spending categories.
- Compare significant spending changes with previous months when sufficient comparable data exists.
- Identify budgets approaching or exceeding their limits.
- Receive practical recommendations supported by recorded financial activity.
- Reopen previously generated reports without running AI again.

Each report should contain:

- 1. **Monthly Overview:** Actual income, expenses, and budget performance.
- 2. **Spending Patterns:** Main spending categories and notable historical changes, where supported.
- 3. **Budget Concerns:** Limits approaching or exceeding their thresholds.
- 4. **Positive Observations:** Areas where spending remained controlled or improved.
- 5. **Recommendations:** Practical actions with supporting spending or budget evidence.

Reports must separate calculated statistics from AI interpretation. Advanced forecasting, investment recommendations, and professional financial advice are not required.

3.2 Should Have: Recommended Features

These features improve financial awareness, usability, and the overall value of SmartWallet.

A. Spending Insights

The application should provide additional information about spending patterns. Users should be able to:

- Identify their highest spending categories.
- Compare expenses between different months.
- Observe changes in spending habits.

- Review income and expense trends over time.

Insights should be based on recorded financial data and clearly distinguish actual spending from planned budgets.

B. Budget Alerts

The platform should alert users when their spending approaches or exceeds a defined budget. For example:

- A category reaches 80% of its monthly budget.
- A category exceeds its spending limit.
- Total monthly spending exceeds the overall budget.

Alerts should help users recognize potential overspending without requiring constant manual budget checks.

C. Savings Progress Visualization

The application should provide visual progress indicators for savings goals. Users should be able to understand:

- How much they have already saved.
- How much remains to reach the target.
- Their progress as a percentage of the goal.
- Whether the goal has been completed.

D. Receipt History

Users should be able to associate uploaded receipts with their corresponding expense transactions. When available, users should be able to reopen the receipt image to verify transaction information. Receipt files should remain private and accessible only to the account owner.

3.3 Could Have: Optional Features

The following improvements offer opportunities for creativity and additional technical exploration.

- **Recurring Transactions:** Allow users to define repeated income or expenses, such as salaries, rent, or subscriptions.
- **Advanced Spending Analysis:** Identify unusual spending patterns or significant changes between months.
- **Financial Data Export:** Allow users to download their transaction history or financial summaries.
- **Savings Recommendations:** Suggest realistic saving opportunities based on recorded spending habits and financial goals.

Students are encouraged to propose other features that improve financial organization, expense tracking, or budget awareness.

Additional functionality should remain consistent with the application's purpose as a personal finance management tool.

4. AI Integration

4.1 Purpose

AI in SmartWallet serves two purposes: simplifying expense recording through receipt OCR and categorization, and interpreting backend-calculated monthly financial statistics to explain spending patterns and suggest practical budgeting improvements.

Manually recording purchases can become repetitive, particularly for users who frequently make small purchases. SmartWallet addresses this challenge by combining OCR and AI-assisted information processing. OCR is responsible for recognizing text from receipt images, while AI helps interpret the extracted information and identify relevant transaction details.

AI should simplify transaction recording and help users understand their financial activity, while keeping users in control and distinguishing calculated statistics from AI interpretations.

4.2 Expected AI Workflow

The receipt processing functionality should follow these steps:

- **1. Receipt Upload:** The user uploads a photo of a shopping receipt.
- **2. Image Processing:** The application prepares the image for text recognition when necessary.
- **3. OCR Extraction:** The system recognizes the text printed on the receipt.
- **4. Information Identification:** The extracted text is analyzed to identify the merchant, transaction date, total amount, and other relevant information.
- **5. Category Suggestion:** AI suggests an appropriate expense category based on the recognized merchant or purchased items.
- **6. User Validation:** The user reviews the extracted fields, corrects mistakes, and selects or changes the suggested category.
- **7. Transaction Creation:** The confirmed information is saved as an expense transaction.
- **8. Financial Updates:** The new transaction is reflected in the user's financial dashboard, budget calculations, and transaction history.

The OCR and AI functionality should be integrated directly into the normal expense creation process.

Monthly Financial Analysis Workflow

- 1. The user selects a month.
- 2. Express.js retrieves recorded transactions, budgets, and savings information.
- 3. The backend calculates actual financial statistics and supported historical comparisons.
- 4. The prepared statistics are passed to the AI module of the Express.js backend.
- 5. AI interprets spending patterns, highlights observations, and suggests evidence-based improvements.
- 6. Express.js validates the generated report.
- 7. The application displays and saves the report for later review.

4.3 Expected AI Capabilities

The AI integration should be capable of:

- Processing recognized receipt text.
- Identifying relevant transaction information from unstructured text.
- Distinguishing the final total from subtotals, discounts, and other amounts when possible.
- Recognizing merchant names and purchase dates.
- Suggesting expense categories based on receipt content.
- Returning structured information that the application can validate.
- Handling incomplete, unclear, or partially recognized receipts.
- Allowing users to correct extracted information before saving.

For monthly analysis, AI must interpret backend-calculated statistics, identify supported spending patterns and budget concerns, explain the evidence behind recommendations, and acknowledge insufficient history.

For example, a user uploads a supermarket receipt containing several grocery items and a total of 235 MAD. The system should identify the merchant, purchase date, and total amount, then suggest the "Groceries" category. The user should be able to review the result, correct any errors, and confirm the expense.

4.4 Technical Considerations

Receipt OCR and monthly AI financial analysis must be implemented within the Express.js backend, as dedicated service modules (for example, an OCR module that reads the receipt image, and an AI module that builds prompts, calls the chosen AI API or model, and validates the structured response). No separate AI backend is required. Financial totals, percentages, budget usage, and historical comparisons must be calculated by Express.js before AI interpretation.

Students should consider the following constraints:

- Receipt images may be blurry, rotated, incomplete, or poorly illuminated.
- Receipts may contain several monetary values, including subtotals, taxes, discounts, and final totals.
- OCR results may contain spelling mistakes or incorrectly recognized numbers.
- The system should avoid inventing missing transaction information.
- Uncertain or missing fields should be clearly presented for manual correction.
- Suggested expense categories should correspond to categories available to the user.
- Extracted information must be validated before a transaction is created.
- Uploaded receipts may contain sensitive financial information and should be handled securely.
- The application should support manual expense creation when OCR or AI processing is unavailable.
- The AI solution should be achievable using free APIs, open-source models, or suitable free-tier services.

Monthly financial analysis must follow these reliability rules:

- Financial amounts, totals, percentages, and comparisons are calculated by the backend, never invented by AI.
- Conclusions and recommendations must reference recorded spending patterns, budgets, or savings information.
- Insufficient historical data must be acknowledged rather than presented as invented trends.
- Spending changes must not be automatically called positive or negative without context.
- Reports must distinguish actual statistics from AI-generated explanations and provide general budgeting suggestions, not investment or professional financial advice.
- Invalid reports or AI failures must not affect transactions, budgets, savings records, or ordinary financial dashboards.

Students may explore open-source OCR solutions available for Node.js, such as Tesseract.js, or OCR and vision APIs with a free tier, alongside suitable AI models for information extraction and categorization. These technologies are suggestions rather than mandatory implementation choices.

The required AI functionality does not depend on connecting to banking services or processing financial payments.

5. Technology Stack and Deployment

5.1 Required Technology Stack (MERN)

The project must be built exclusively with the MERN stack. No other framework, language, or database is required or expected.

Component	Technology
Database	MongoDB (with Mongoose as the ODM)
Backend	Node.js with Express.js (main application logic, OCR and AI integration)
Frontend	React.js (functional components and hooks, with React Router for navigation)
AI Functionality	Receipt OCR, expense categorization, and AI monthly financial analysis, integrated through the Express.js backend
Version Control	Git and GitHub

MongoDB is well suited to SmartWallet: transactions, categories, monthly budgets, savings goals, receipts, and generated reports can be stored in separate collections linked by references, while flexible documents suit OCR results and AI reports. Students should justify their data modeling decisions; evaluation focuses on the quality of their reasoning and implementation. Financial calculations should remain consistent and reliable (for example, by using MongoDB aggregation pipelines for totals and budget usage).

The Express.js backend is responsible for the main application logic, authentication, authorization, transaction management, database operations, receipt OCR, expense categorization, and monthly financial analysis. The React frontend consumes the backend REST API.

Students are responsible for designing the database, organizing application architecture, and selecting appropriate approaches for image handling and for structuring the OCR and AI modules

within the backend.

5.2 Deployment

The completed SmartWallet application must be deployed and accessible online. The frontend, Express.js backend (including OCR and AI integration), and MongoDB database must communicate correctly in the deployed environment.

Suggested deployment services include:

Component	Suggested Service
React Frontend	Vercel or Netlify
Express.js Backend (including OCR and AI integration)	Railway or Render
MongoDB Database	MongoDB Atlas

These providers are suggestions rather than mandatory choices. Students may select alternative hosting services that satisfy the project requirements, preferably using available free tiers or free resources.

The deployed application must support manual financial tracking, receipt processing, and generation and retrieval of monthly AI reports.

6. Expected Deliverables

The final project submission should include:

- A functional full-stack SmartWallet application built with the MERN stack.
- Secure user authentication and personal account management.
- Income and expense transaction management.
- Transaction categorization with default and custom categories.
- Monthly budget creation and monitoring, and personal savings goal tracking.
- A financial dashboard with relevant summaries and spending visualizations.
- Receipt image upload and OCR processing.
- AI-assisted transaction information extraction and expense categorization.
- User review and correction of extracted receipt information.
- AI-generated monthly financial reports.
- Historical spending comparisons when sufficient data exists.
- Recommendations grounded in recorded financial data.
- Saved monthly financial analysis history.
- Validation and error handling for generated financial reports.
- Persistent database integration using MongoDB.
- Source code available in a GitHub repository.

- A README containing the project overview, installation steps, configuration, and deployment instructions.
- Sample transactions, categories, budgets, savings goals, and receipt images for demonstration.
- A live deployed application accessible online.
- A final presentation and demonstration of the application's main workflows.

Example End-to-End Scenario

- **1. User Setup:** Yassine, a young professional seeking to better understand his monthly spending and save for a new laptop, creates a SmartWallet account and selects MAD as his preferred currency.
- **2. Income & Budget Setup:** At the beginning of the month, he records his salary as an income transaction and sets a monthly spending budget of 4,000 MAD, along with category-specific budgets (1,500 MAD for groceries, 600 MAD for transportation).
- **3. Savings Goal Initialization:** He defines an 8,000 MAD savings goal for his laptop and records an initial contribution of 1,000 MAD.
- **4. Receipt Upload & AI Processing:** After purchasing groceries, he uploads a photo of his supermarket receipt instead of manually entering the expense. The OCR service extracts the merchant name, purchase date, and total amount (285 MAD), while AI suggests the "Groceries" category.
- **5. Review & Automated Updates:** Yassine reviews and confirms the extracted details and saves the transaction. The application automatically updates his expense history, groceries budget, and monthly financial dashboard.
- **6. Dashboard Monitoring & Adjustments:** Later in the month, Yassine checks his dashboard, notices transportation expenses approaching their planned limit, reviews recent transactions, and adjusts his spending decisions.
- **7. Goal Tracking:** He records another contribution toward his laptop savings goal and verifies the remaining balance needed to reach his target.
- **8. Monthly Financial Analysis:** At the end of the month, Yassine requests an AI financial report. Express.js calculates his spending statistics and compares them with previous months when sufficient records exist. AI summarizes relevant patterns and budget concerns.
- **9. Review and Recommendations:** Yassine reviews any spending increases or exceeded budget limits, considers the report's evidence-based suggestions, and plans adjustments for the next month.

This scenario connects transactions, receipt OCR, budgeting, savings tracking, and monthly AI analysis grounded in recorded financial data.

7. Evaluation Criteria

The project will be evaluated according to the following criteria:

Criterion	Description
Functional Completeness	Correct implementation of the required features and main personal finance management workflows.
Frontend and User Experience	Interface quality, responsiveness, ease of use, and clarity of financial information and interactions.
Backend and Database	Quality of API development, financial calculations, data management, authentication, authorization, and business logic, including the justification and implementation quality of the MongoDB data model.
AI Integration	Reliability of receipt OCR, financial information extraction, and expense categorization; quality of monthly financial analysis, evidence-based observations, practical recommendations, accurate backend calculations, and handling of insufficient data.
Code Quality	Organization, readability, maintainability, and appropriate technical decisions.
Deployment	Accessibility and correct integration of all application components, including OCR and AI functionality.
Team Collaboration	Effective collaboration, Git usage, individual contributions, and understanding of the implemented solution.
Final Demonstration	Ability to demonstrate complete workflows, explain technical choices, and present a functional application.

The expected outcome is a coherent, functional, and practical personal finance management platform that demonstrates the ability to combine full-stack MERN development, OCR, and artificial intelligence to simplify expense tracking, improve budget awareness, and support personal savings goals.